

UNIVERSITY OF RWANDA
COLLEGE OF SCIENCE AND TECHNOLOGY
SCHOOL OF ICT

Computer and Software Engineering Department
Nyarugenge Campus

MODULE CODE / TITLE: WEB DESIGN

Project Technical Report
Web Technology – TECHNICAL ASSESSMENT 2026

Project Title	BizFlow – Daily Stock & Finance Management Web Application
Student Name	UMWANAMWIZA Serge
Registration Number	225045359
Programme	BSc Computer Engineering – Year 1
Academic Year	2025 – 2026
Submission Date	19 th July 2026
Supervisor	Dr Eric HITIMANA

© 2026 University of Rwanda – College of Science and Technology

List Of abbreviations

ICT	Information and Communication Technology
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
UI	User Interface
UX	User Experience
RWF	Rwandan Franc
CRUD	Create, Read, Update, Delete
API	Application Programming Interface
ERD	Entity-Relationship Diagram
MoMo	Mobile Money
SPA	Single Page Application

Chapter 1. Introduction

1.1 Historical Background of the Case Study

Small and medium-scale businesses in Rwanda — particularly those in retail, wholesale distribution, and bar-restaurant sectors — have historically managed their daily stock, credits, debts, and cash flows through paper-based ledgers or basic spreadsheet tools such as Microsoft Excel. While these methods provided a functional baseline, they suffered from significant drawbacks including manual calculation errors, difficulty in tracking historical data across multiple days, inability to derive issued quantities automatically, and no mechanism for real-time financial summary.

The Rwandan government's push for digital transformation under Vision 2050 and the Smart Rwanda Master Plan has created growing demand for affordable, locally-relevant digital tools tailored to small business realities. BizFlow was conceived in this context — as a direct response to the operational challenges observed in everyday Rwandan businesses, particularly those managing fast-moving consumer goods (FMCGs) such as beverages, where inventory must be tracked daily with precision.

The inspiration for the data model came directly from existing Excel-based workflows used by a beverage distributor in Rwanda, where stock sheets were maintained per day (one sheet per day in a monthly workbook) and issued quantities were derived using the formula: $\text{Issued Qty} = \text{Previous Day Stock} + \text{Received} - \text{Closing Stock}$. BizFlow digitises this workflow into a responsive single-page web application accessible from any browser without installation.

1.2 Problem Statement

The primary operational challenges faced by the target businesses include:

- Inventory discrepancies arising from manual counting errors and the inability to automatically calculate issued quantities across multiple product lines.
- Lack of a daily carry-forward mechanism for credits (money owed by clients) and debts (money owed to suppliers), leading to lost payment records between days.
- No centralised daily summary showing total capital, stock value, outstanding receivables, and money on hand in one view.
- Administrative difficulty in managing multiple user accounts and recovering forgotten credentials without a dedicated administrator tool.
- Time-consuming reconciliation of piece-level sales (individual bottles vs. full cases) against case-level inventory.

These gaps result in financial leakage, unreconciled balances, and time wasted on manual data entry — problems that a well-designed web application can solve efficiently.

1.3 Objectives of the Project

Main Objective:

To design and implement a secure, browser-based daily stock and financial management web application tailored to Rwandan small businesses, enabling accurate inventory tracking, automated issued-quantity calculation, and consolidated financial reporting.

Specific Objectives:

- Implement a daily sheet system where products, their opening stock, received quantities, closing stock, and auto-calculated issued quantities are tracked per day using the formula: $\text{Issued} = \text{Opening} + \text{Received} - \text{Closing Stock}$.
- Develop a global product registry that carries all products across every day without re-entry, eliminating duplicate data and reducing input effort.
- Build a credits and debts management module with a full payment history log per creditor/debtor, allowing payments to be recorded for any past or current day.
- Create a financial summary view that calculates total capital as: $\text{Capital} = \text{Stock Value} + \text{Credits Outstanding} + \text{Money on Hand} - \text{Debts Outstanding}$.
- Implement a secure multi-user registration and login system with an administrator panel for credential recovery and account management.
- Deliver a clean, professional UI using only HTML, CSS, and vanilla JavaScript — no backend required — with all data persisted in browser localStorage.

1.4 Expected Outcomes

- A fully functional single-page web application (stock.html) allowing daily inventory tracking with automatic issued-quantity computation.
- Persistent global product, credit, and debt data that carries forward across all daily sheets without re-entry.
- A daily financial dashboard summarising stock value, issued value, piece revenue, outstanding credits/debts, expenses, money on hand, and total capital.
- A multi-day payment tracking system for credits and debts showing the payment history log per client/party with date, amount, and notes.
- A secure admin panel (admin.html) enabling the administrator to view all registered user accounts and passwords, reset credentials, and manage accounts.
- A responsive, professional design compatible with desktop and mobile browsers without installation.

1.5 Project Rationale (Importance to the Community)

BizFlow addresses a real, observed gap in how Rwandan SMEs manage their daily financial operations. Its importance to the community is multifold:

- **Economic accuracy:** Automating issued-quantity calculations using a proven formula eliminates manual errors that cause stock losses worth thousands of RWF daily.
- **Financial inclusion:** By building on browser localStorage rather than requiring a paid server or cloud subscription, BizFlow is accessible to low-resource businesses that cannot afford SaaS tools.
- **Debt management discipline:** The carry-forward reste system ensures that outstanding balances from clients and suppliers are never lost between days, improving cash-flow visibility.
- **Skill demonstration:** The project demonstrates the practical application of web technologies (HTML5, CSS3, JavaScript ES6+) learned in the WEB DESIGN module to a real-world Rwandan business problem.
- **Scalability pathway:** The architecture — modular JS functions, localStorage persistence, and clean separation of daily snapshots from global data — provides a natural migration path to a cloud-backed solution in a future phase.

Chapter 2. System Analysis and Design

2.1 System Analysis

2.1.1 Intended Users of the Project

BizFlow is designed for two primary user classes:

- **Business Operators (Regular Users):** Small business owners or their designated data-entry staff who use the application daily to record stock movements, payments received, expenses, and money on hand. They interact with the Daily Sheet, Inventory, Credits, Debts, Expenses, and Money tabs.
- **Administrator:** A single privileged user (the business owner or system manager) who accesses the Admin Panel (admin.html) to manage user accounts, view or reset passwords, and delete accounts. The admin does not use the daily operational tabs.

Secondary beneficiaries include auditors or accountants who may review the exported CSV reports for monthly reconciliation purposes.

2.1.2 Intended System Partners

While BizFlow v1 operates entirely client-side (no server), its data model is designed to integrate with the following potential partners in future phases:

- **Rwanda Revenue Authority (RRA):** The TIN number field collected during registration anticipates future tax-reporting integration.
- **Mobile Money Providers (MTN MoMo, Airtel Money):** The money-on-hand section tracks MoMo balances explicitly, aligning with the dominant payment channel in Rwanda.
- **Bank of Kigali / Equity Bank:** Bank balance tracking prepares the ground for open-banking API integration.
- **Huawei Cloud / AWS:** The modular JS architecture can be migrated to a Node.js + cloud database backend with minimal refactoring.

2.2 System Design

2.2.1 High-Level Architecture

BizFlow follows a client-side Single Page Application (SPA) architecture:

Layer	Technology	Role
Presentation	HTML5 + CSS3	UI structure and styling (tabs, cards, tables, modals)
Application Logic	Vanilla JavaScript ES6+	CRUD operations, formula engine, render functions
Data Persistence	Browser localStorage	Per-user keyed JSON storage for all app data
Authentication	localStorage session	Login state management; admin via

		sessionStorage
File Distribution	Static HTML files	No server required; open login.html to start

The data model uses a two-tier global/snapshot pattern:

- Global stores (products, credits, debts) persist across all days and are keyed by user email.
- Daily snapshots (snap_YYYY-MM-DD) store per-day changes (closing stock, received qty, payments) keyed by product name or entity ID.

The issued quantity formula engine derives $\text{Issued/Qty} = \text{Opening} + \text{Received} - \text{Closing Stock}$ for each product, matching the Excel-based formula: $=-B2:B24+C2:C24--\text{Sheet30!B2:B24}$

2.2.2 UI Design and Flowcharts

The application is structured into six main tabs accessible from a persistent left sidebar:

- Daily Sheet – date selector, day switcher, add new day button
- Inventory – global product table with per-day stock tracking
- Credits – client credit tracking with payment history log
- Debts – supplier/party debt tracking with payment history log
- Expenses – daily expense categorisation
- Money – cash and mobile money balance entry
- Summary – consolidated capital and financial overview

Each tab follows a consistent card-based layout: a statistics bar at the top, a data entry form, and a live table below. The colour scheme uses a dark green primary (#0A7C59), amber accents (#F4A024), and a dark navy header (#111827).

2.2.3 Database Design with ERD and Relationships

Since BizFlow uses localStorage (client-side key-value storage), the data model is described as a logical ERD mapped to JSON structures:

Entity / Key	Fields	Relationships
users[]	id, name, email, business, tin, password	Root entity; one user → many days
products[]	name, qty(opening), pu, pieces, pprice	Global; referenced by all daily snapshots
snap_DATE{}	productName → {stockQty, received, pieces}	One snapshot per day per user
credits[]	id, name, frw, payments[], lastUpdated	Global; payments[] logged per day
debts[]	id, name, frw, payments[], lastUpdated	Global; payments[] logged per day

day_DATE.expenses[]	id, cat, frw, desc	Per-day, per-user
day_DATE.money{}	cash, momo, code, mokash, bank	Per-day, per-user

Key relationships: one User has many Days; one Day references global Products via snapshots; Credits and Debts each maintain an embedded payments log that links each payment to a specific day key.

Chapter 3. Implementation

3.1 Introduction

BizFlow was implemented entirely using client-side web technologies: HTML5 for structure, CSS3 (with CSS custom properties) for styling, and vanilla JavaScript ES6+ for all application logic and data management. No frameworks, build tools, or server infrastructure were required. The application consists of four inter-linked HTML files: login.html (user authentication), register.html (new account creation), stock.html (main application), and admin.html (administrator control panel).

All data is persisted in the browser's localStorage under user-specific keys prefixed with bf_<email>_, ensuring data isolation between multiple users on the same device. The application implements a complete daily workflow cycle: open the app, select or create a day, record inventory movements, log payments, enter expenses and money on hand, and review the consolidated financial summary.

3.2 Application Screenshots and Discussion

3.2.1 User Registration Page (register.html)

The registration page collects the user's full name, business name, TIN number, email address, and password. Client-side validation ensures all required fields are completed before saving the new account to localStorage. The page features the BizFlow branding with a dark navy header panel and the green accent colour scheme consistent across all pages. A duplicate email check prevents the same account from being registered twice.

3.2.2 Login Page (login.html)

The login page authenticates users against the stored accounts in localStorage. Upon successful authentication, the user's email and business name are saved to a loggedInUser session key and the user is redirected to stock.html. The login page also features a subtle "Admin Panel" link at the bottom for administrator access, and a "Register" link for new users.

3.2.3 Admin Panel (admin.html)

The admin panel is protected by separate hardcoded credentials (username: admin, password: bizflow2024). Once authenticated, the administrator sees a statistics bar showing the total number of registered accounts, distinct businesses, and accounts without a TIN number. The main table displays all registered users with their email, business name, TIN, and password. Passwords can be revealed individually using the eye toggle button, or all at once using "Show / Hide All". The administrator can reset any user's password via a modal dialog, delete accounts, and export all user data as a CSV file. This page was built specifically to address the "forgotten password" problem that would otherwise require direct localStorage inspection.

3.2.4 Daily Sheet – Inventory Tab (stock.html)

The Inventory tab is the core of the application. Products are added once to a global registry and automatically appear on every day without re-entry. Each product row displays the opening stock (carried from the previous day's closing stock), a received quantity input, an editable closing

stock/qty input, the auto-calculated issued quantity, the price per unit, the stock value in RWF, and the issued value in RWF. For piece-level tracking, the opening pieces, a closing pieces input, and derived piece-issued count and value are shown. A duplicate product guard prevents the same product from being added twice.

The issued quantity is computed in real time using the formula: $\text{Issued/Qty} = \text{Opening Stock} + \text{Received} - \text{Closing Stock}$, which mirrors the Excel formula `=B2:B24+C2:C24--Sheet30!B2:B24` used in the reference spreadsheet. A footer row shows column totals. The table header notes that Stock/Qty and Rem Pieces columns are editable (marked with )

3.2.5 Credits Tab

The Credits tab tracks money owed by clients. Each credit entry stores the client name, the total amount owed, and a payments log array. Payments can be added for the current day using the quick "+ Pay" inline button, or for any past or future day using the " Past" button which opens a modal with a day selector dropdown. Clicking a client name expands an inline payment history row showing all payments with their date, amount, optional note, and a delete button. The Reste (remaining balance) column shows the total owed minus all payments recorded up to and including the currently viewed day, so the balance automatically reflects the correct outstanding amount when switching between days. A " Paid" badge appears and the row dims when the reste reaches zero.

3.2.6 Debts Tab

The Debts tab mirrors the Credits tab in structure but tracks money owed to external parties (suppliers, landlords, etc.). The same payment history mechanism, day-selective payment modal, inline expansion, and carry-forward reste logic applies. The colour coding uses red/amber for debt figures to visually distinguish them from the green credit figures.

3.2.7 Summary Tab

The Summary tab provides a consolidated financial overview. At the top, a highlighted Capital card displays the computed capital: $\text{Capital} = \text{Stock Value} + \text{Credits Outstanding} + \text{Money on Hand} - \text{Debts Outstanding}$. The card breaks down each component. Below, four cards show Inventory details, Credits & Debts breakdown, Expenses by category, and Money on Hand by payment channel (Cash, MoMo, Code, MoKash, Bank). The capital figure is coloured green when positive and red when negative.

Chapter 4. Conclusion and Recommendation

4.1 Conclusion

BizFlow has been successfully implemented as a fully functional single-page web application that addresses the core daily financial management needs of Rwandan small businesses. Approximately 95% of the defined objectives were achieved within the project timeline:

- Daily stock tracking with automatic issued-quantity calculation using the Excel-derived formula
- Global product registry carrying all products across days without re-entry
- Credits and debts management with full payment history and day-selective payment recording
- Capital computation combining stock value, outstanding credits, money on hand, and debts
- Multi-user authentication with a dedicated admin panel for credential management
- Responsive design compatible with both desktop and mobile browsers
- Future: Cloud backup / multi-device synchronisation (not yet implemented)

The application demonstrates how modern web technologies can solve real business problems without requiring expensive infrastructure, making it immediately deployable in low-resource environments across Rwanda.

4.2 Recommendations

To the Lecturer

The WEB DESIGN module provided an excellent foundation in HTML, CSS, and JavaScript. It is recommended that future iterations of the module include sessions on data persistence strategies (localStorage vs. IndexedDB vs. cloud APIs) and client-side security considerations, as these are critical for any real-world web application.

To the School

The project timeline, while achievable for a focused SPA, was tight given that students also manage coursework across other modules simultaneously. It is recommended that project briefs be distributed at the start of the semester to allow iterative development rather than concentrated delivery near the deadline.

To Business Partners / Owners

The current version of BizFlow stores all data in the browser's localStorage, which means data is tied to a single device and browser. Business owners are strongly advised to use the CSV export function regularly to back up their data, and to avoid clearing browser data or switching browsers without first exporting. A cloud-synced version is planned for a future phase.

To Users

Users should attend a brief orientation session (30–60 minutes) to understand the daily workflow: create a day, enter closing stock figures (not opened bottles), and let the system calculate what was issued. The distinction between "closing stock" as an input and "issued qty" as a derived output is the most important concept to grasp for accurate records.

To Beneficiaries / Customers

Customers who are tracked in the Credits module can expect more accurate and transparent balance records. Disputes over outstanding amounts can be resolved by reviewing the dated payment history log, which provides an auditable record of every payment made and on which day it was recorded.

Appendix

A. GitHub Repository

The complete source code for BizFlow is available at:

<https://github.com/Sergeu250/Business-solution.git>

Repository access: Public

Default branch: main

Files included: login.html, register.html, stock.html, admin.html

B. Technology Stack Summary

Technology	Usage in BizFlow	
HTML5	Document structure, forms, semantic markup	
CSS3	Custom properties, grid/flex layout, animations, responsive design	
JavaScript ES6+	Application logic, DOM manipulation, localStorage CRUD, formula engine	
Browser localStorage	Client-side data persistence for all user and business data	
sessionStorage	Admin authentication state within a browser session	
Google Fonts (Inter)	UI typography for modern, readable interface	
Times New Roman	Document typography (this report)	

C. Data Formula Reference

The core inventory formula implemented in BizFlow, derived from the client's Excel workbook:

BizFlow JS: `issued = Math.max(0, opening + received - closingStock)`

Capital formula: `capital = stockValue + creditsOutstanding + moneyOnHand - debtsOutstanding`